MongoDB + Express

Setup & Installation

Install Packages

```
npm init -y
npm install express mongoose dotenv
```

Project Structure

```
my-app/
├── server.js    ├── .env
├── models/User.js
└── routes/users.js
```

.env File

```
MONGO_URI=mongodb://localhost/mydb
PORT=5000
```

server.js — Setup

```
const express  = require('express');
const mongoose = require('mongoose');
require('dotenv').config();
const app = express();
app.use(express.json());
```

Connect to MongoDB

```
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('Connected'))
  .catch(err => console.log(err));
```

Start Server

```
app.listen(process.env.PORT, () =>
  console.log('Server running'));
```

Mongoose Schema & CRUD

Define Schema (models/User.js)

```
const mongoose = require('mongoose');
const UserSchema = new
mongoose.Schema({
  name:  { type: String, required:
true },
  email: { type: String, unique: true
},
  age:   { type: Number, default: 0 },
  createdAt:{ type:Date,
default:Date.now}
});
module.exports =
  mongoose.model('User', UserSchema);
```

CREATE — POST

```
router.post('/', async (req, res) => {
  const user = new User(req.body);
  await user.save();
  res.status(201).json(user);
});
```

READ — GET All / By ID

```
router.get('/', async (req, res) => {
  res.json(await User.find());
});
router.get('/:id', async (req,res) =>
{
  res.json(await User.findById(
    req.params.id));
});
```

UPDATE — PUT

```
router.put('/:id', async (req, res) =>
{
  const u = await
User.findByIdAndUpdate(
    req.params.id,
req.body,{new:true});
  res.json(u);
});
```

DELETE

```
router.delete('/:id',async(req,res)=>{
  await User.findByIdAndDelete(
    req.params.id);
  res.json({ message: 'Deleted' });
});
```

Routes, Queries & Tips

Register Routes (server.js)

```
const userRoutes =
  require('./routes/users');
app.use('/api/users', userRoutes);
```

Mongoose Query Methods

- **Filter:** User.find({ age:{$gt:18} })
- **FindOne:** User.findOne({ email })
- **ById:** User.findById(id)
- **Select:** .select('name email')
- **Sort:** .sort({ name: 1 })
- **Paginate:** .limit(10).skip(5)

Error Handling Middleware

```
app.use((err, req, res, next) => {
  res.status(500).json({
    error: err.message });
});
```

Async/Await Pattern

```
try {
  const data = await Model.find();
  res.json(data);
} catch (err) {
  res.status(500).json({ err });
}
```

HTTP Status Codes

- 200 OK — success GET / PUT
- 201 Created — success POST
- 400 Bad Request — invalid data
- 404 Not Found — missing resource
- 500 Server Error — exception

Mongoose Operators

- $gt, $lt, $gte, $lte — compare
- $in: [a,b] — match array
- $or, $and — logical
- $set — partial update

React & Angular

Web Programming Exam Cheatsheet · Page 2 of 2

React — Basics & Hooks

Create & Start

```
npx create-react-app my-app
cd my-app && npm start
```

Functional Component

```
function Welcome({ name }) {
  return <h1>Hello, {name}!</h1>;
}
export default Welcome;
```

JSX Rules

- One root element or <>…</>
- className not class
- camelCase events: onClick
- JS in { } | self-close

useState

```
import { useState } from 'react';
const [count, setCount] =
useState(0);
<button
onClick={()=>setCount(c=>c+1)}>
  {count}</button>
```

useEffect

```
useEffect(() => {
  fetchData();           // on mount
  return () => cleanup();// unmount
}, [dep]); // [] = once only
```

Fetch Data Pattern

```
const [data,setData] = useState([]);
useEffect(() => {
  fetch('/api/users')
    .then(r=>r.json())
    .then(d=>setData(d));
}, []);
```

useRef

```
const ref = useRef(null);
<input ref={ref} />
ref.current.focus();
```

React — Props, Router & Context

Props

```
<Card title='Hi' count={5} />
function Card({ title, count }) {
  return <p>{title}: {count}</p>;
}
```

Lists & Conditional

```
{items.map((x,i)=><li
key={i}>{x}</li>)}
{ok && <Dashboard />}
{err ? <Err /> : <Content />}
```

Forms

```
const [val,setVal] = useState('');
<input value={val}

onChange={e=>setVal(e.target.value)}/>
```

React Router v6

```
npm install react-router-dom
import { BrowserRouter, Routes,
  Route, Link } from 'react-router-
dom';
<BrowserRouter><Routes>
  <Route path='/' element={<Home/>}/>
  <Route path='/about'
    element={<About/>}/>
</Routes></BrowserRouter>
```

useNavigate & useParams

```
const nav = useNavigate();
nav('/home');
const { id } = useParams();
```

Context API

```
const Ctx = createContext();
<Ctx.Provider value='dark'>
  <App /></Ctx.Provider>
// child: useContext(Ctx)
```

Lifecycle Summary

- Mount: useEffect(()=>{}, [])
- Update: useEffect(()=>{}, [dep])
- Unmount: return ()=> cleanup()
- Memo: useMemo / useCallback

Angular

Install & Create

```
npm install -g @angular/cli
ng new my-app
cd my-app && ng serve
```

Generate

```
ng g component home
ng g module app-routing
```

Component

```
@Component({
 selector:'app-home',
 templateUrl:'./home.html',
 styleUrls:['./home.css']
})
export class HomeComponent {
 title='Hello'; count=0;
 inc(){ this.count++; }
}
```

Template Syntax

- {{ expr }} — interpolation
- [prop]='x' — property bind
- (event)='fn()' — event bind
- [(ngModel)]='x' — two-way
- *ngIf / *ngFor

Routing

```
const routes:Routes=[
 {path:'',component:HomeComp},
 {path:'**',redirectTo:''}];
<router-outlet></router-outlet>
<a routerLink='/about'>Go</a>
this.router.navigate(['/home']);
```

Reactive Forms

```
// import ReactiveFormsModule
form=new FormGroup({
 name:new FormControl('',
  Validators.required),
 email:new FormControl('')});
onSubmit(){
  console.log(this.form.value);}
```

Lifecycle & Pipes

- ngOnInit() — after render
- ngOnChanges() — input changed
- ngOnDestroy() — cleanup

```
{{ price | currency:'USD' }}
{{ date  | date:'short' }}
{{ name  | uppercase }}
```